SELECTED ENGINEERING PORTFOLIO / OCTOBER 2026

Robotics AI & Systems Integration

Brandon Gerbasi

Computer Science B.S. candidate · Southern New Hampshire University
Palo Cedro, California · Seeking Austin-area opportunities and relocation

The work I enjoy most

"The part I enjoy most is final integration - taking AI, software, sensors, compute, electronics, and hardware and bringing the complete machine to life."

I build across software, local AI, electronics, networking and mechanical fabrication. My projects focus on making the interfaces work together, exposing failures clearly, and testing each stage before adding more complexity.

AI & learning

Local Qwen3 inference, speech services, PyTorch, reviewed wake-word data and model evaluation.

Robotics software

C++17, Python, ROS2, state estimation, telemetry replay, computer vision and safety logic.

Physical integration

ESP32-S3, Raspberry Pi, sensors, KiCad PCB design, UAV components, CAD and 3D printing.

Systems & deployment

Linux, NVIDIA/CUDA, authenticated services, systemd, diagnostics, testing and recovery.

Field experience

Tesla manufacturing, apprentice electrical work, property maintenance and independent field services.

Current direction

Robotics systems integration, physical AI, embedded/edge systems, automation and field reliability.

2 Linux GPU nodes

Local AI host and separate speech worker.

3 capstone enhancements

Design, algorithms and SQLite persistence.

40 × 30 mm carrier

Two-layer KiCad design submitted for fabrication.

512-294-0580 | brandon.gerbasi@gmail.com
Portfolio / GitHub / LinkedIn

IN THIS PORTFOLIO

02 Otto Core · 03 Mobile & glove · 04 PCB design · 05 UAV & propulsion
06 Robotics software · 07 CS 499 capstone · 08 Field background & PLC learning

Independent and academic projects are distinguished from professional employment. Project status reflects the documented checkpoint of October 12, 2026.

PROJECT 1 / LOCAL AI & SYSTEMS SOFTWARE

Otto: distributed local AI & robotics assistant

Working two-node platform · Ongoing independent development

Purpose: build a private, local-first assistant that combines language and speech services, document retrieval, explicit memory and guarded tool execution across Linux/NVIDIA computers.

AI host · RTX 3090

Qwen3-30B-A3B inference, Python orchestration, local state and Piper speech output.

Speech worker · RTX 3060

faster-whisper speech recognition through an authenticated service on a separate Linux/NVIDIA host.

What I built

Modular services. Separated language-model inference, speech recognition, text-to-speech, prompts, tools and state management. Integrated managed startup, endpoint checks and bounded request handling rather than treating each model as an isolated demo.

Grounded knowledge and explicit memory. Added local SQLite FTS5 and embedding-based retrieval, source-grounded answers, structured memory and separate project/task/reminder records. Durable changes require explicit intent and confirmation.

Scoped execution. Built deterministic tool-domain selection, bounded tool loops and confirmation gates for protected actions. Hardware-control placeholders remain separate from working system and document tools.

Health and recovery. Added host/service checks, incident histories, diagnostic reports and allowlisted service recovery with cooldowns and attempt limits. Owner-operated recovery tooling captures application state and verifies backup integrity.

Wake-word dataset engineering & evaluation

Built an owner-reviewed audio workflow with immutable originals, hash-tracked intake, duplicate checks, normalization, audit records and session-aware train/validation/test exports. The v1.5.1 review workflow seals per-record decisions before applying them to the dataset.

CNN and temporal candidates were trained and evaluated in the development lab. Candidates that did not meet readiness criteria were not promoted into the live voice path. More recordings and a larger count alone are not treated as proof of a reliable wake detector.

Current engineering focus

Continue voice reliability, representative evaluation and field-client integration. Context size, speed and test counts are configuration-specific; this portfolio does not present an older benchmark as a current live-system guarantee.

TECHNOLOGIES Python · Linux · Qwen3 · faster-whisper · Piper · SQLite · CUDA · systemd

Physical boundary: Otto does not yet operate the complete UAV or a deployed smart-home control system.

OTTO EXTENSION / EMBEDDED & NETWORK INTEGRATION

Otto Mobile & the gesture-control glove

Controlled cellular voice pilot completed · Wearable revision in development

Purpose: provide a screen-free interface to Otto using fingertip contacts, a wearable controller, a portable Linux client and private earbud playback. The development spans firmware, audio devices, networking and physical packaging.

Part of the system	Implemented pilot role
Fingertip interface	ESP32-S3 firmware interprets thumb/finger contact gestures and sends control events over bonded BLE.
Portable client	Raspberry Pi 5 receives gesture events and records through the Pi-connected ReSpeaker microphone array.
Transport	Cellular data and private WireGuard networking carry requests through restricted SSH/RPC operations.
Backend	Otto Main and Node 2 provide the request-processing and speech-service path.
Playback	The reply returns to the Pi and plays through the enrolled Bluetooth earbuds.

What the controlled demonstration established

Completed a battery-powered cellular request/reply/playback cycle. The retained trial records include matching audio-integrity evidence, playback acknowledgment and cleanup across the participating endpoints. Component checks were followed by an end-to-end test rather than assuming that connectivity alone meant the interaction worked.

The accepted pilot included a deterministic time request. It demonstrates the portable control, transport and audio loop; it is not presented as a free-form language-model performance benchmark.

The next physical revision

The newer glove/cuff work includes two-layer leather construction, short nickel-silver fingertip contact plates, prepared wiring and removable electronics. A custom carrier PCB brings the contact, microphone and battery interfaces together while keeping the XIAO ESP32-S3 separate.

The cuff microphone and new carrier are a later hardware revision. Their audio path, assembled-board behavior, enclosure fit and wearable acceptance still need their own testing; the earlier ReSpeaker-based pilot is not used as proof that this revision works.

Prototype boundary

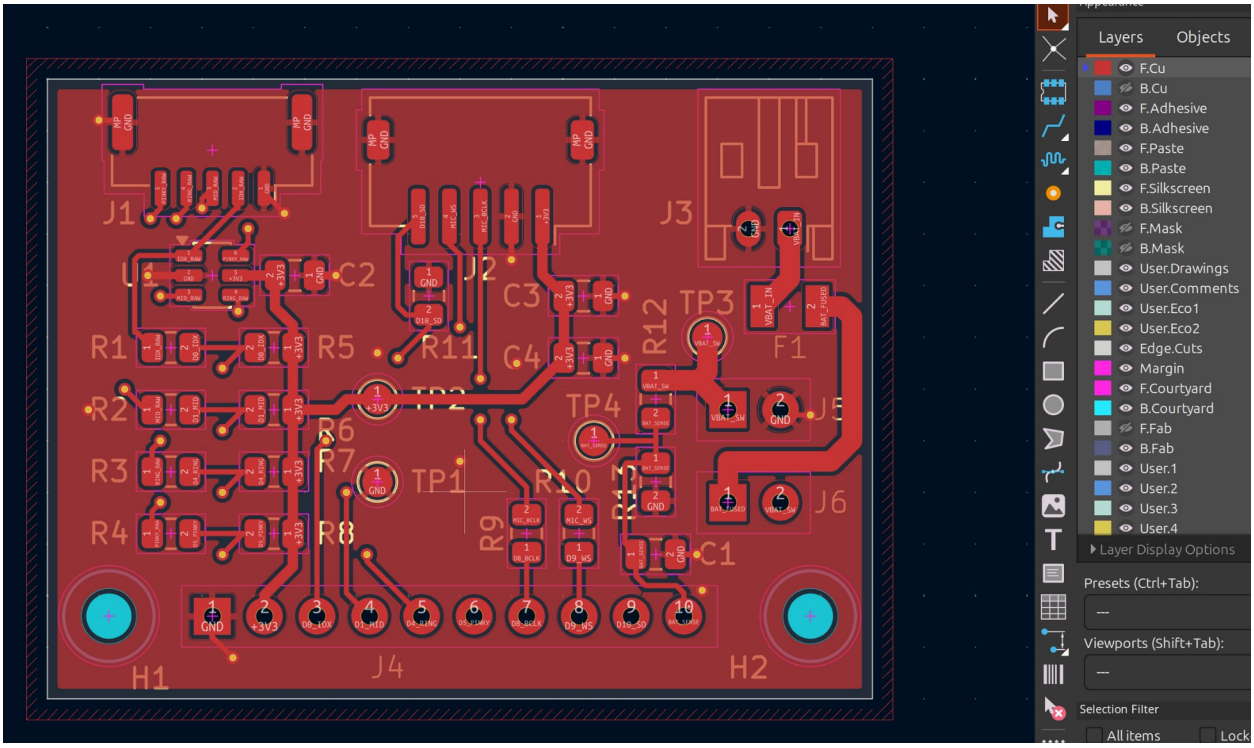
The demonstrated milestone is a cellular-data voice interface to Otto, not a finished telephone/SMS replacement or a validated all-day roaming device. Carrier assembly and the new cuff-audio path remain pending.

HARDWARE DESIGN / OTTO WEARABLE INTERFACE

Glove carrier Rev A

KiCad design completed and submitted to PCBWay · Assembly and testing pending

Designed a 40 × 30 mm, two-layer carrier for the glove contact inputs, external microphone interface and battery connections. The XIAO ESP32-S3 and microphone breakout remain separate modules, connected to the carrier.



Original KiCad layout from the build handoff. This is CAD evidence, not a photograph of assembled hardware.

Circuit scope

Four contact inputs, static-discharge protection, external microphone connections, fused/switched battery path, battery-voltage sensing and test points.

Completed design work

Schematic capture, custom wire-pad/fuse footprints, placement, two-layer routing, ground pours, stitching vias and fabrication-layer/drill review.

Recorded design-rule check

Zero active errors, zero active warnings and zero unconnected items. Three individually documented footprint-name exclusions remain; electrical testing is separate.

Manufacturing & next steps

Exported Gerber and drill files and submitted the design to PCBWay. Awaiting boards; assemble and electrically validate the first board before treating the revision as working hardware.

ENGINEERING CONTRIBUTION Took a wearable-interface need through schematic, footprint, layout and manufacturing handoff while preserving a clear first-board acceptance stage.

View the glove carrier design case study

PROJECT 2 / PHYSICAL ROBOTICS & PROTOTYPING

Custom field robotics UAV platform

Airframe fabrication completed through multiple revisions · Integration in progress

Purpose: design a custom aerial platform that brings propulsion, power, flight-control hardware, sensing, telemetry and edge computing together. The main UAV and compact ducted-impeller experiment are separate projects.

Subsystem	Project work and present scope
Mechanical platform	Custom CAD, 3D-printed airframe components and mounts; component layout, payload placement and repeated design revisions.
Power & propulsion	6S batteries, high-current distribution, ESCs and motors selected for the platform; wiring and complete-system integration remain active work.
Flight & communication	Flight Core V2, GPS and control/telemetry hardware form the flight-system plan. Complete-aircraft validation remains a separate milestone.
Sensing & edge compute	Jetson Orin Nano, LiDAR, range, thermal and visual sensing are staged for later integration and perception work.
Ground/operator software	Separate Python telemetry/replay tools, C++ state estimation and ROS2 safety/logging demonstrations support the development process.

Separate experiment: ducted impeller & Coanda-inspired geometry

Designed and printed propulsion housings and geometry iterations, including stator and outlet revisions, and explored their behavior through bench prototypes. The work is a physical design experiment rather than a demonstrated hovering aircraft or a validated aerodynamic improvement.

Completed addition: digital load-cell mounting fixtures

Modeled and 3D-printed multiple mounting designs for digital load-cell sensors to support a developing thrust-measurement setup. This provides a tangible test-fixture contribution alongside the propulsion designs.

Next measured milestone

Assemble and calibrate the thrust setup, record repeatable measurements, and compare design iterations under documented conditions. Printed fixtures are complete; calibrated thrust, efficiency, endurance and a measured Coanda-related benefit are not yet reported.

TECHNOLOGIES CAD · 3D printing · Sensors · Propulsion · Power distribution · Flight hardware

SUPPORTING PROJECTS / ROBOTICS SOFTWARE

Robotics software & validation tools

Independent software demonstrations · Separate from complete-aircraft integration

These projects exercise state, sensing, fault handling and operator feedback in smaller, repeatable systems. They complement the physical builds without implying that every software function has been deployed on the full UAV.

Project / stack	Implemented behavior
C++ Robot State Estimator **C++17 · CMake · GTest**	Tracks position, heading and velocity; handles GPS loss, reports confidence and produces repeatable CSV/Markdown/JSON outputs and fault-test artifacts.
UAV Telemetry Ground Station **Python · Replay · Log analysis**	Simulates or replays flight-state data, detects unsafe and high-current conditions, and produces operator-facing reports for normal and mixed-fault scenarios.
ROS2 Perception & Safety Demo **ROS2 Jazzy · C++**	Separates range sensing, IMU input, safety control and logging. Demonstrates CLEAR, SLOW_DOWN, STOP_OBSTACLE and STOP_TILT state behavior.
Vision-Based Object Tracker **Python · OpenCV**	Demonstrates real-time visual tracking as a stand-alone perception project and a foundation for future robotics integration.
Edge-Deploy Safety Monitor **Python · Automated tests**	Evaluates sensor/system conditions and produces explicit safe-state outputs using repeatable scenarios.

Engineering emphasis

Observable behavior

Logs, structured reports, confidence values and explicit state transitions make faults easier to inspect and reproduce.

Clear integration limits

A passing software scenario is kept separate from live sensor acceptance, flight-controller behavior and full-aircraft readiness.

Additional academic software: full-stack web application

Built an Angular administrator interface and a Node.js/Express REST API with MongoDB persistence, JWT authentication and protected create/update/delete operations. Used Postman to exercise authenticated and unauthenticated endpoints and documented request flow and application architecture.

This academic application complements the robotics work with API design, persistence, authentication and client/server integration.

GitHub profile and project repositories

ACADEMIC WORK / SOUTHERN NEW HAMPSHIRE UNIVERSITY

CS 499: C++ capstone

Three enhancements completed and submitted · Final ePortfolio presentation in progress

Artifact: the CS 410 Project2.cpp investment-company client-service application. It authenticates a user, displays client information and changes a client's selected service. The capstone develops the same artifact across three categories rather than substituting unrelated examples.

01 / Software engineering & design

Refactored the single-file program into focused C++17 classes: Client, InputValidator, AuthenticationService, ClientManager and AppController. Separated keyboard input, application flow, authentication and client-data operations; kept data access controlled and added repeatable CMake builds.

Testing addressed failed logins, invalid input, partially accepted numbers, client updates and clean exits when input ends. The improvement is in organization, reliability and testability while preserving the original application's purpose.

02 / Algorithms & data structures

Added stable client IDs and unordered_map storage for average O(1) ID lookup and ID-based updates. Sorted views copy records to a vector and use std::sort in O(n log n) time; service filtering scans the records in O(n).

Documented the trade-off between fast keyed access and ordered display. Kept search, sort, filter and update behavior in ClientManager while the menu remains in AppController.

03 / Databases

Completed SQLite persistence with prepared statements, constraints, transactions and an audit log. The database enhancement builds on the modular and ID-based versions, adding durable storage and a record of changes rather than relying only on in-memory client records.

Completed submission stages	Remaining publication work
Enhancement one: Module Three. Enhancement two: Module Four. Enhancement three: Module Five.	Organize the final enhanced-source files, narratives and self-assessment in the public ePortfolio; incorporate final feedback as needed.

ACADEMIC SCOPE The authentication implementation is an educational demonstration, not a production credential-security solution. Completing the enhancements is not the same as final course or degree conferral.

CS 499 ePortfolio section / Original C++ artifact

PROFESSIONAL BACKGROUND / LEARNING & DIRECTION

Field experience & controls development

Professional roles below are separate from the independent engineering projects

Background	Relevant experience
Tesla Gigafactory Nevada **Production Associate · 2026**	Battery-module manufacturing with safety, quality and traceability requirements; cross-training across production areas and team recognition for performance.
Apprentice electrical & maintenance work	Industrial/commercial apprentice electrical work and property maintenance, including hands-on troubleshooting, repairs, wiring and equipment work.
Independent field services **Six years of experience**	Managed field assignments, route planning, deadlines and records independently; handled difficult customer interactions with clear communication and professional judgment.
Embodied-AI data collection **Temporary project**	Performed repeatable physical tasks while wearing sensing equipment to produce human-demonstration data. Confidential project details are omitted.

PLC fundamentals & home-lab development

Completed free introductory Rockwell Automation training and worked through start/stop control logic. Purchased PLC components for hands-on practice and small automation projects. The current milestone is completed introductory study and a home lab being developed, not industrial PLC commissioning experience or a professional certification.

Technical focus

Software & data

C++17, Python, ROS2, OpenCV, SQLite, CMake, Git, API integration and automated testing.

Systems & hardware

Linux, NVIDIA/CUDA, systemd, ESP32-S3, Raspberry Pi, BLE, KiCad, CAD/3D printing, wiring and soldering.

Education & direction

Southern New Hampshire University · Computer Science B.S. candidate, expected 2026. Current capstone work includes the three completed CS 499 enhancements described on page 7.

Seeking Austin-area opportunities in robotics systems integration, embedded/edge systems, automation, field deployment and reliability. Currently based in Palo Cedro, California, with a firm intention to relocate to the Austin area as soon as possible. Open to onsite and field work.

What I bring to an integration team

Hands-on troubleshooting across software, power, communications and mechanics; independent field responsibility; and a habit of building, testing and documenting unfamiliar systems until their behavior is understood.

brandon.gerbasi@gmail.com / Portfolio / GitHub / LinkedIn